ANÁLISIS TÉCNICO · SIN EJECUCIÓN

Arquitectura AWS FleetLogix

Recursos, funciones Lambda, decisiones de diseño y costos

Archivos analizados: A4-06_aws_setup.py · A4-lambda_functions.py

Servicios: API Gateway · Lambda · DynamoDB · S3 · RDS PostgreSQL · SNS · IAM

Entregable: AWS_Análisis_Arquitectura.pdf — Avance 4, curso de Ciencia de Datos

1. Introducción

Este documento analiza, sin ejecutar en AWS, la arquitectura propuesta para el Avance 4 de FleetLogix. Se describe qué recursos crean los scripts **A4-06_aws_setup.py** y **A4-lambda_functions.py**, qué función cumple cada uno dentro del flujo de la aplicación, y se profundiza en dos piezas clave solicitadas por la consigna: la función Lambda *verificar-entrega* y la función *migrar_datos_postgresql()*. Cierra con las decisiones de arquitectura tomadas y una estimación de costos mensuales en capa gratuita/uso bajo.

2. Diagrama de arquitectura

El siguiente diagrama resume el flujo: la app móvil se comunica con API Gateway, que enruta las solicitudes hacia las tres funciones Lambda. Estas leen y escriben en DynamoDB, publican alertas por SNS y se apoyan en una base PostgreSQL migrada a RDS y un bucket S3 para históricos.

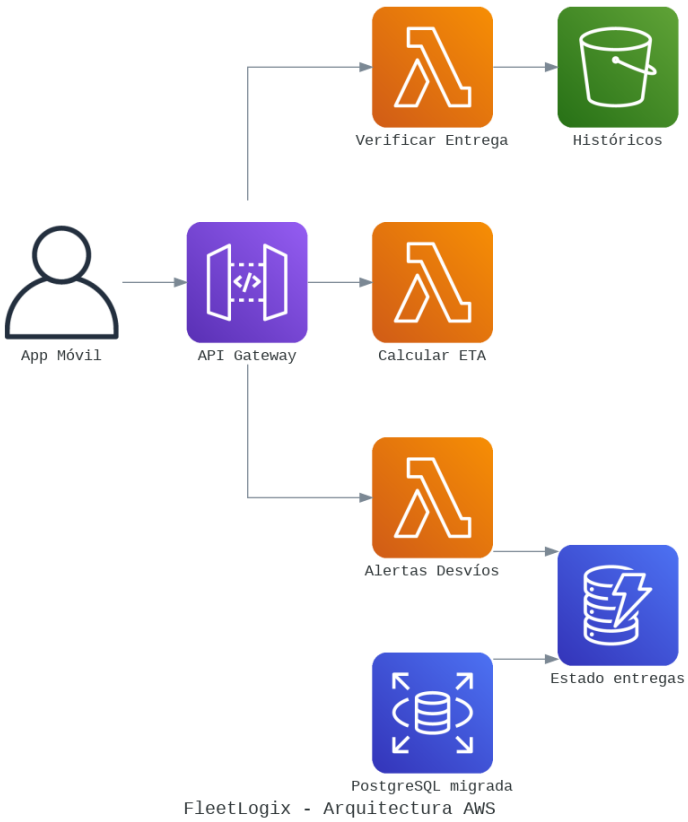


Figura 1. Arquitectura AWS de FleetLogix (aws_architecture_diagram.png).

3. Recursos creados y funcionalidad

3.1 A4-06_aws_setup.py

Este script no procesa datos de la flota: aprovisiona la infraestructura donde luego corren las Lambdas y vive la base de datos. Cada función crea (o detecta si ya existe) un recurso puntual de AWS:

Función	Recurso AWS	Qué hace
crear_rds_postgresql()	RDS · PostgreSQL	Instancia db.t3.micro (free tier), motor PostgreSQL 15.4, 20 GB gp2, backups automáticos diarios (retención 7 días) y ventana de mantenimiento semanal. Queda accesible públicamente para simplificar el desarrollo.
crear_s3_bucket()	S3	Bucket fleetlogix-data con 4 'carpetas' lógicas (raw-data, processed-data, backups, logs) y una regla de ciclo de vida que mueve raw-data a Glacier a los 90 días para abaratar el almacenamiento histórico.
crear_tablas_dynamodb()	DynamoDB	4 tablas en modo PAY_PER_REQUEST (on-demand, sin aprovisionar capacidad): deliveries_status (PK delivery_id), vehicle_tracking (PK vehicle_id + SK timestamp), routes_waypoints (PK route_id) y alerts_history (PK vehicle_id + SK timestamp).
configurar_backups_automaticos()	RDS	Genera un snapshot manual inicial de la instancia RDS, además de confirmar que el backup automático (7 días) quedó activo desde la creación.
migrar_datos_postgresql()	Local → RDS	No migra datos directamente: escribe en disco el script migrate_to_rds.sh que, al ejecutarse aparte, hace pg_dump de la base local y la restaura en RDS. Se detalla en 4.2.
crear_rol_iam_lambda()	IAM	Rol FleetLogixLambdaRole con trust policy para el servicio lambda.amazonaws.com y 4 políticas administradas adjuntas: ejecución básica de Lambda, y acceso completo a DynamoDB, S3 y SNS.

El main() orquesta todo en secuencia (RDS → S3 → DynamoDB → backups → script de migración → rol IAM) y al final persiste un resumen en aws_config.json, útil para que otros scripts o el equipo sepan qué nombres de recursos quedaron creados.

3.2 A4-lambda_functions.py

Este archivo no crea infraestructura: define la lógica de negocio de tres funciones Lambda que se despliegan manualmente en la consola de AWS (copiando cada función y renombrándola a lambda_handler, como indica la guía). Cada una resuelve un caso de uso distinto de la operación de la flota:

Función	Disparador (trigger)	Qué hace
lambda_verificar_entrega	API Gateway POST /deliveries/verify	Consulta deliveries_status en DynamoDB por delivery_id y responde si la entrega ya fue marcada como 'delivered'. Se detalla en la sección 4.1.

Función	Disparador (trigger)	Qué hace
lambda_calcular_eta	EventBridge cada 5 minutos	A partir de ubicación actual, destino y velocidad, estima distancia (aproximación tipo Haversine simplificada) y hora estimada de llegada; guarda el resultado en vehicle_tracking.
lambda_alerta_desvio	Kinesis Stream (actualizaciones GPS)	Compara la posición actual del vehículo contra los waypoints de routes_waypoints. Si la distancia mínima supera 5 km, publica una alerta en SNS y la registra en alerts_history.

4. Profundización en dos funciones clave

4.1 lambda_verificar_entrega

Es la Lambda más simple de las tres, pensada como endpoint de consulta síncrono detrás de API Gateway. Su lógica, paso a paso:

- **Entrada:** recibe un evento con delivery_id (obligatorio) y opcionalmente tracking_number.
- **Validación:** si falta delivery_id, corta de inmediato y responde 400 (Bad Request) sin tocar la base de datos.
- **Consulta:** abre la tabla DynamoDB deliveries_status y hace un get_item por clave primaria (delivery_id) — es una lectura puntual, no un scan, por lo que es económica y rápida.
- **Entrega encontrada:** arma is_completed comparando item['status'] == 'delivered' y devuelve 200 con el estado, el tracking number guardado y la fecha de entrega si existe.
- **Entrega no encontrada:** responde 404 indicando que ese delivery_id no existe en la tabla.
- **Errores inesperados:** cualquier excepción (por ejemplo, falta de permisos IAM o tabla inexistente) se captura y se traduce en un 500 con el mensaje de error — este es justamente el comportamiento que la guía muestra como 'error esperado' cuando se prueba la función antes de correr aws_setup.py, ya que la tabla todavía no existe.

En términos de diseño, la función es *stateless* y no escribe nada: solo lee. Esto la hace segura de invocar repetidas veces (idempotente) y barata de operar, ya que DynamoDB en modo PAY_PER_REQUEST solo cobra por esa lectura puntual.

4.2 migrar_datos_postgresql()

A diferencia de lo que su nombre sugiere, esta función **no migra datos en el momento en que se la invoca**: genera un script de shell (migrate_to_rds.sh) que el usuario debe ejecutar manualmente en un paso posterior. Es una decisión deliberada: separar la generación de la receta de migración de su ejecución, para poder revisarla antes de correrla contra una base real.

El script generado hace 3 cosas, en orden:

1. **Exportar** — pg_dump de la base local (fleetlogix) a un archivo fleetlogix_dump.sql.
2. **Crear** — se conecta al endpoint de RDS y ejecuta CREATE DATABASE para crear la base destino.
3. **Restaurar** — vuelve a conectarse a RDS, ahora contra la base recién creada, y ejecuta psql con el dump para cargar todos los datos.

Puntos a tener en cuenta:

- El RDS_ENDPOINT queda hardcodeado como placeholder (fleetlogix-db.xxxx.us-east-1.rds.amazonaws.com); recién se conoce el endpoint real una vez que la instancia RDS terminó de crearse (~10 minutos, según el propio main()), por lo que ese valor debería actualizarse antes de correr el script.
- Es una migración de una sola vez (one-shot): no hay lógica incremental ni de sincronización continua entre la base local y RDS.
- Al ser un .sh separado, requiere tener pg_dump y psql instalados en la máquina que lo ejecuta, y que esa máquina tenga salida de red hacia RDS (por eso RDS se configuró como PubliclyAccessible=True en este entorno de desarrollo).
- No maneja contraseñas de forma segura: la guía asume que se ingresan de forma interactiva o vía variables de entorno de psql, ya que no están escritas en el script.

5. Decisiones de arquitectura

API Gateway como punto único de entrada

Centraliza autenticación, throttling y ruteo hacia las 3 Lambdas, evitando exponer cada función por separado y facilitando agregar versionado o autorización más adelante.

DynamoDB en modo on-demand (PAY_PER_REQUEST)

Para un proyecto académico con tráfico variable e impredecible, evita sobre-aprovisionar capacidad de lectura/escritura y sus costos fijos; se paga solo por lo que efectivamente se usa.

Triggers distintos por función

verificar-entrega es síncrona (API Gateway, la app espera respuesta inmediata); calcular-eta es periódica (EventBridge cada 5 min, no necesita respuesta en tiempo real); alerta-desvio es reactiva a streaming (Kinesis), pensada para procesar actualizaciones GPS casi en tiempo real. Cada trigger se eligió según la urgencia y el patrón de datos de entrada.

RDS PostgreSQL en paralelo a DynamoDB

Se mantiene PostgreSQL para los datos relacionales/analíticos ya modelados en el pipeline ETL (Avances anteriores), mientras que DynamoDB atiende consultas operativas de baja latencia (estado de una entrega puntual, último tracking de un vehículo). Es un patrón de persistencia poliglota: cada base para lo que mejor resuelve.

S3 con lifecycle a Glacier

Los datos crudos pierden valor de consulta frecuente después de los primeros meses; moverlos a Glacier a los 90 días reduce el costo de almacenamiento sin perder el histórico.

Rol IAM único para las 3 Lambdas

Simplifica el setup inicial (un solo rol con 4 políticas administradas) a costa de otorgar más permisos de los estrictamente necesarios por función; en un entorno productivo conviene separar roles con permisos mínimos por Lambda (principio de menor privilegio).

6. Estimación de costos AWS (uso bajo / capa gratuita)

Estimación orientativa para un entorno de desarrollo con tráfico bajo, según lo documentado en la guía y los precios públicos de AWS para us-east-1. Los valores reales dependen del volumen efectivo de uso.

Servicio	Costo estimado	Comentario
RDS PostgreSQL (db.t3.micro, 20 GB)	~USD 15 / mes	El recurso más caro de la arquitectura; no está cubierto por la capa gratuita fuera del primer año de una cuenta nueva.
DynamoDB (4 tablas, on-demand)	USD 0 – 2 / mes	Con bajo volumen de lecturas/escrituras, prácticamente gratis; capa gratuita perpetua de 25 GB y cierta cantidad de solicitudes.
S3 (bucket + lifecycle a Glacier)	< USD 1 / mes	Costo mínimo mientras el volumen de históricos sea chico; Glacier abarata aún más los datos de más de 90 días.

Servicio	Costo estimado	Comentario
Lambda (3 funciones)	USD 0 / mes	Capa gratuita perpetua de 1 millón de invocaciones y 400.000 GB-segundo de cómputo mensuales; un proyecto académico no debería superarla.
API Gateway	USD 0 – 1 / mes	Capa gratuita de 1 millón de llamadas/mes durante el primer año; luego se cobra por millón de solicitudes.
SNS (alertas)	USD 0 / mes	Capa gratuita de 1 millón de publicaciones mensuales, muy por encima del uso esperado.

Total estimado: aproximadamente USD 15–18/mes, prácticamente todo explicado por la instancia RDS. Si el proyecto necesitara reducir costos, la opción más directa sería apagar la instancia RDS fuera de las sesiones de trabajo o migrarla a Aurora Serverless v2, que escala a cero cuando no hay actividad.

7. Conclusión

La arquitectura combina servicios administrados y de pago por uso (Lambda, DynamoDB on-demand, S3, SNS) con un único recurso de costo fijo (RDS), lo que mantiene el proyecto dentro de un presupuesto acotado para fines académicos. Las tres funciones Lambda cubren los tres tipos de trigger más comunes en sistemas de flota (síncrono, programado y por streaming), y el script de setup deja documentado en aws_config.json todo lo aprovisionado, lo que facilita auditar o reproducir el entorno sin necesidad de ejecutarlo para este análisis.